

Module-2

Django ORM

Django ORM (Object Relational Mapper) is a powerful feature of Django that allows developers to interact with relational databases using Python code instead of writing SQL queries manually. It acts as a bridge between Python objects and database tables.

In Django:

- Model Class → Represents a database table
- Model Field → Represents a table column
- Object → Represents a row in the table

Django automatically converts Python code into SQL queries.

Django ORM is Used

To Avoid Writing SQL Queries

Developers do not need to write complex SQL statements. Django automatically converts Python code into SQL queries internally.

To Simplify Database Operations

Common operations like inserting, retrieving, updating, and deleting data can be performed using simple Python methods.

Example:

- Insert → `.create()`
- Retrieve → `.all()`, `.filter()`
- Update → `.update()`
- Delete → `.delete()`

To Improve Code Readability

By: Rakshitha P, Asst-Prof, Mepartment of Computer Applications, NMAMIT, Nitte

Python code is easier to understand compared to SQL queries. ORM makes the code clean and maintainable.

To Ensure Security

Django ORM protects applications from **SQL Injection attacks** by automatically escaping user inputs.

To Support Multiple Databases

Django ORM works with:

- SQLite
- MySQL
- PostgreSQL

Developers can change the database without rewriting queries.

Advantages of Django ORM

1. Faster Development

Reduces development time because built-in methods handle most database operations.

2. Database Abstraction

Developers work with Python objects instead of database tables directly.

3. Automatic Relationship Handling

Django ORM easily manages:

- One-to-One relationships
- One-to-Many (ForeignKey)
- Many-to-Many

No need to manually write JOIN queries.

4. Migration Support

By: Rakshitha P, Asst-Prof, Mepartment of Computer Applications, NMAMIT, Nitte

When models are changed, Django automatically creates migration files to update the database safely.

Commands:

- makemigrations
- migrate

5. Query Optimization

Supports filtering, ordering, aggregation, and complex queries efficiently.

6. Clean Project Structure

Keeps database logic inside models.py, maintaining proper separation of concerns.

Relational Databases

A **relational database** is a type of database that stores data in the form of **tables**. Each table consists of **rows** (records) and **columns** (fields). It follows a structured format where data is organized logically, making it easy to store, retrieve, and manage information efficiently.

Relational databases are based on the **relational model**, which was proposed by E. F. Codd. Data in different tables can be connected using relationships such as primary keys and foreign keys

Structure of Relational Database

- **Table** → Collection of related data
- **Row (Record)** → Individual entry in a table
- **Column (Field)** → Attribute of the data
- **Primary Key** → Uniquely identifies each record
- **Foreign Key** → Creates relationship between two tables

Example

If we have two tables:

Student Table

- id (Primary Key)
- name
- department

Department Table

- dept_id (Primary Key)
- dept_name

If dept_id is used in Student table as a foreign key, it creates a relationship between Student and Department tables.

Features of Relational Databases

1. Data is stored in tabular format
2. Uses Structured Query Language (SQL)
3. Supports relationships between tables
4. Ensures data integrity and consistency
5. Supports normalization to reduce redundancy

Advantages

- Easy to understand and manage
- Maintains data accuracy
- Reduces duplication of data
- Supports complex queries
- Suitable for large-scale applications

Examples of Relational Databases

- MySQL
- PostgreSQL
- SQLite
- Oracle Database

Creating Models and Fields in Django

In Django, a model is a Python class that represents a database table. Each attribute inside the model class represents a field, which corresponds to a column in the database table. Django automatically creates the table structure based on the model definition.

Models are defined inside the `models.py` file of an app.

Creating a Model

To create a model, we import models from Django and define a class that inherits from `models.Model`.

Example:

```
from django.db import models
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()  
    email = models.EmailField()
```

Here:

- Student → Table name
- name, age, email → Columns in the table

After creating the model, run:

```
python manage.py makemigrations  
python manage.py migrate
```

Field Types in Django

Field Name	Purpose	Syntax	Example
CharField	Stores short text (name, title, etc.)	models.CharField(max_length=...)	name = models.CharField(max_length=100)
TextField	Stores large text (description, comments)	models.TextField()	description = models.TextField()
IntegerField	Stores integer numbers	models.IntegerField()	age = models.IntegerField()
FloatField	Stores decimal numbers	models.FloatField()	price = models.FloatField()
BooleanField	Stores True or False values	models.BooleanField()	is_active = models.BooleanField()

DateField	Stores date (YYYY-MM-DD)	models.DateField()	dob = models.DateField()
DateTimeField	Stores date and time	models.DateTimeField()	created_at = models.DateTimeField(auto_now_add=True)
EmailField	Stores and validates email addresses	models.EmailField()	email = models.EmailField()
FileField	Used to upload files	models.FileField(upload_to='folder/')	document = models.FileField(upload_to='docs/')
ImageField	Used to upload images	models.ImageField(upload_to='folder/')	photo = models.ImageField(upload_to='images/')
AutoField	Auto-incrementing primary key	models.AutoField(primary_key=True)	id = models.AutoField(primary_key=True)
ForeignKey	Creates One-to-	models.ForeignKey(Model, on_delete=...)	dept = models.ForeignKey(Departm

	Many relationship		ent, on_delete=models.CASCADE)
OneToOneField	Creates One-to-One relationship	models.OneToOneField(Model, on_delete=...)	profile = models.OneToOneField(Profile, on_delete=models.CASCADE)
ManyToManyField	Creates Many-to-Many relationship	models.ManyToManyField(Model)	subjects = models.ManyToManyField(Subject)

Model Relationships in Django

In Django, model relationships are used to connect one table with another table in a relational database. These relationships are created using special fields inside models.py. Django automatically manages the database connections using primary keys and foreign keys.

There are three main types of relationships:

- One-to-One
- One-to-Many
- Many-to-Many

One-to-One Relationship

In a **One-to-One** relationship, one record in a table is related to only one record in another table.

Example:

One Student has one ID Card.

Syntax

```
models.OneToOneField(ModelName, on_delete=models.CASCADE)
```

Example

```
from django.db import models
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)
```

```
class IDCard(models.Model):  
    student = models.OneToOneField(Student, on_delete=models.CASCADE)  
    card_number = models.CharField(max_length=20)
```

- Each student gets only one ID card.
- If the student is deleted, the ID card is also deleted (CASCADE).

One-to-Many Relationship

In a **One-to-Many** relationship, one record in a table can be related to multiple records in another table.

Example:

One Department has many Students.

Syntax

```
models.ForeignKey(ModelName, on_delete=models.CASCADE)
```

Example

```
class Department(models.Model):  
    name = models.CharField(max_length=100)
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    department = models.ForeignKey(Department, on_delete=models.CASCADE)
```

Explanation

- One department can have many students.
- Each student belongs to only one department.
- ForeignKey creates the relationship.

Many-to-Many Relationship in Django (Including through)

In Django, a **Many-to-Many** relationship means multiple records in one table can be related to multiple records in another table. Django normally creates an automatic intermediate table. However, if we want to store **extra fields in the relationship**, we use the through option.

Many-to-Many

Used when no extra data is required in the relationship.

Example:

A Student can study many Subjects.

A Subject can have many Students.

Syntax

```
models.ManyToManyField(ModelName)
```

Example

```
from django.db import models
```

```
class Subject(models.Model):  
    name = models.CharField(max_length=100)
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    subjects = models.ManyToManyField(Subject)
```

- Django automatically creates a third table (Student_Subject).
- No need to manually create the intermediate table.

Many-to-Many with through

We use through when we need **additional information** in the relationship.

Example:

A Student enrolls in many Subjects, but we also want to store:

- enrollment_date
- marks
- grade

In this case, the relationship itself has extra data.

Syntax

```
models.ManyToManyField(ModelName, through='IntermediateModel')
```

Example

```
class Student(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
class Subject(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
class Enrollment(models.Model):
```

```
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
```

```
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE)
```

```
    enrollment_date = models.DateField()
```

```
    marks = models.IntegerField()
```

```
class Student(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    subjects = models.ManyToManyField(Subject, through='Enrollment')
```

- Enrollment is the intermediate (bridge) table.
- It stores extra fields like marks and enrollment_date.
- Django will NOT automatically create the third table because we created it manually.

Making Migrations and Applying Them in Django

In Django, **migrations** are used to update the database structure whenever changes are made in models. When we create or modify models, Django does not directly change the database. Instead, it creates migration files, which are instructions to modify the database schema.

What is Migration?

A **migration** is a file that contains instructions for creating or modifying database tables based on changes made in models.py.

Migration helps to:

- Create new tables
- Add or remove fields
- Modify field types
- Delete tables
- Maintain version control of database structure

Step 1: Making Migrations

After creating or modifying models, run:

```
python manage.py makemigrations
```

♦ What This Command Does:

- Checks changes in models.py
- Creates a migration file inside the migrations/ folder
- The file name looks like:
0001_initial.py

This file contains Python code representing database changes.

Step 2: Applying Migrations

After creating migration files, run:

```
python manage.py migrate
```

What This Command Does:

- Reads migration files
- Applies changes to the database
- Creates or updates tables

Now the database is synchronized with models.

Step 3: Register Model in Admin Page

To manage data from the admin panel, we must register the model in `admin.py`.

Open `admin.py`

```
from django.contrib import admin
from .models import Student
admin.site.register(Student)
```

What this does:

- Makes the Student model visible in Django admin panel
- Allows adding, editing, deleting records through UI

Example

Create Model in `models.py`

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Register model in `admin.py`

Run Commands

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Result:

Student table is created in the database.

QuerySet and QuerySet API in Django

In Django, a **QuerySet** is a collection of objects from the database. It represents the result of a database query. QuerySets allow us to read, filter, update, and delete data using Python code instead of writing SQL.

The **QuerySet API** is a set of methods provided by Django to perform database operations.

Creating Objects

Creating means inserting new records into the database table.

Syntax 1: Using create()

```
ModelName.objects.create(field=value)
```

Example

```
Student.objects.create(name="Rama", age=21)
```

Syntax 2: Using save()

```
obj = ModelName(field=value)
```

```
obj.save()
```

Example

```
s = Student(name="Krishna", age=22)
```

```
s.save()
```

Retrieving Data (Fetching Records)

Introduction

Retrieving means getting data from the database.

Method	Purpose	Syntax	Example
all()	Retrieve all records	Model.objects.all()	Student.objects.all()
get()	Retrieve single object	Model.objects.get(condition)	Student.objects.get(id=1)
filter()	Retrieve multiple matching records	Model.objects.filter(condition)	Student.objects.filter(age=21)
exclude()	Exclude specific records	Model.objects.exclude(condition)	Student.objects.exclude(age=21)
first()	First record	Model.objects.first()	Student.objects.first()
last()	Last record	Model.objects.last()	Student.objects.last()
count()	Count records	Model.objects.count()	Student.objects.count()
exists()	Check if records exist	Model.objects.filter(condition).exists()	Student.objects.filter(age=21).exists()
values()	Return data as dictionary	Model.objects.values()	Student.objects.values()
values_list()	Return data as tuple	Model.objects.values_list()	Student.objects.values_list('name')
order_by()	Sort records	Model.objects.order_by('field')	Student.objects.order_b

			y('age')
distinct()	Remove duplicate records	Model.objects.distinct()	Student.objects.distinct()

Filtering Methods

Filtering is used to retrieve data based on conditions.

Methods	Purpose	Syntax	Example
exact	Exact match	field__exact=value	Student.objects.filter(name__exact="Rama")
icontains	Case-insensitive match	field__icontains=value	Student.objects.filter(name__icontains="rama")
contains	Contains text	field__contains=value	Student.objects.filter(name__contains="Ra")
icontains	Case-insensitive contains	field__icontains=value	Student.objects.filter(name__icontains="ra")
gt	Greater than	field__gt=value	Student.objects.filter(age__gt=20)
lt	Less than	field__lt=value	Student.objects.filter(age__lt=25)
gte	Greater than or equal	field__gte=value	Student.objects.filter(age__gte=21)
lte	Less than or equal	field__lte=value	Student.objects.filter(age__lte=22)
in	Match in list	field__in=[list]	Student.objects.filter(age__in=[20,21])

range	Between range	field__range=(a,b)	Student.objects.filter(age__range=(20,25))
startswith	Starts with	field__startswith=v alue	Student.objects.filter(name__startswith="R")
endswith	Ends with	field__endswith=v alue	Student.objects.filter(name__endswith="a")
isnull	Check NULL	field__isnull=True	Student.objects.filter(age__isnull=True)

Updating Data

Updating means modifying existing records in the database.

Method 1: Using update() Syntax:

Model.objects.filter(condition).update(field=value)

Example:

```
Student.objects.filter(id=1).update(age=23)
```

Updates directly in database

Does not call save() method

Method 2: Using save() (Object Method)

Syntax:

```
obj = Model.objects.get(condition)
```

```
obj.field = new_value
```

```
obj.save()
```

Example:

```
s = Student.objects.get(id=1)
```

```
s.age = 24
```

```
s.save()
```

Calls save()

Good when updating single object

Deleting Data

Deleting means removing records from the database.

Method 1: Delete Using get() (Single Object)

Used when deleting one specific record.

Syntax:

```
obj = ModelName.objects.get(condition)
```

```
obj.delete()
```

Example:

```
student = Student.objects.get(id=1)
```

```
student.delete()
```

- `get(id=1)` retrieves one student.
- `delete()` removes that student from the database.

Method 2: Delete Single Object

Syntax:

```
obj = Model.objects.get(condition)
```

```
obj.delete()
```

Example:

```
s = Student.objects.get(id=1)
```

```
s.delete()
```

Method 3: Delete Multiple Objects

Syntax:

```
Model.objects.filter(condition).delete()
```

Example:

```
Student.objects.filter(age=20).delete()
```